REPUBLIC OF CAMEROON
PEACE-WORK-FATHERLAND
UNIVERSITY OF BUEA
BUEA. SOUTH-WEST REGION

RÉPUBLIQUE DU CAMEROUN
PAIX-TRAVAIL-PATRIE
UNIVERSITÉ DE BUEA
BUEA, RÉGION DU SUD-OUEST

FACULTY OF ENGINEERING AND TECHNOLOGY
DEPARTMENT OF COMPUTER ENGINEERING, SE
CEF440: INTERNET PROGRAMMING

TASK 5: UI Design and Implementation

Report by: GROUP 11

Department: Computer Engineering

CHECKAR – SMART CAR DIAGNOSTICS APPLICATION

Course Facilitator:

Dr. NKEMENI VALERY

Date Issued:

8TH JUNE 2026

Academic Year: 2025/2026

Name	Matricule
FUNWI KELSEA NDOHNWI	FE23A066
NGADANG ASSONFACK P. ESTELLE	FE23A109
ATEH SWIRRI FOFANG	FE23A018
TABOT GHISLAIN OROCK	FE23A146

Table of Content

Executive Summary

The rapid advancement of vehicle technology has introduced increasingly complex diagnostic systems that are often difficult for ordinary drivers to understand. To address this challenge, the CheckAR Smart Car Diagnostics Application was designed as an intelligent mobile solution capable of identifying car faults through dashboard scanning and engine sound analysis.

This report presents the User Interface (UI) Design and Frontend Implementation of the CheckAR application. The report focuses on how visual design principles, user experience strategies, and frontend technologies were applied to create an intuitive, responsive, and accessible mobile application.

The application was designed using modern mobile UI standards with emphasis on simplicity, usability, and efficiency. The interface allows users to:

- Scan vehicle dashboard warning lights
- Record and analyze engine sounds
- View diagnostic reports
- Track maintenance history
- Locate nearby mechanics
- Receive maintenance reminders

The report further explains the visual identity of the application, including typography, colors, layout systems, iconography, navigation structure, and responsive design strategies. In addition, it discusses frontend implementation using Flutter, state management strategies, user interaction flows, accessibility standards, and optimization techniques.

The UI was carefully developed to support both technical and non-technical users while ensuring smooth interaction across Android and iOS platforms.

1. Introduction

1.1 Project Context

Modern vehicles contain numerous electronic systems and sensors that generate warning indicators and engine feedback signals. Many car users lack the technical knowledge needed to interpret these signals correctly, resulting in delayed maintenance and increased repair costs.

The CheckAR mobile application was developed to simplify vehicle diagnostics using Artificial Intelligence (AI), computer vision, and sound analysis technologies. The system enables users to diagnose faults using their smartphone camera and microphone without requiring advanced mechanical knowledge.

1.2 UI Design Objectives

The major objectives of the UI design include:

- Create a simple and intuitive user experience
- Reduce diagnostic complexity for non-technical users
- Improve accessibility and ease of navigation
- Ensure consistency across all application screens
- Build user trust through professional visual design
- Support fast interaction and real-time feedback
- Provide responsive layouts for different screen sizes

1.3 Target User Analysis

1.3.1 Primary Users (Car Users – Non-Technical)

These users include everyday drivers with little or no mechanical experience. The interface was designed to:

- Use simple language
- Provide visual guidance
- Minimize technical complexity
- Offer clear diagnostic explanations

1.3.2 Secondary Users (Mechanics as Car Users)

Professional mechanics can use the application for faster preliminary diagnostics and maintenance tracking.

Their needs include:

- Fast access to reports
- Detailed issue descriptions
- Maintenance history tracking
- Mechanic recommendations

1.3.3 Administrative Users

Administrators manage:

- User accounts
- Machine learning model updates
- System monitoring
- Performance analytics

The admin interface focuses on functionality, organization, and system monitoring efficiency.

2. App Identity Design

2.1 Brand Name Rationale

The name “CheckAR” combines:

- “Check” representing vehicle inspection and diagnostics
- “AR” representing advanced smart technology and augmented assistance

The brand name communicates innovation, intelligence, and reliability.

2.2 Logo Design and Visual Metaphor

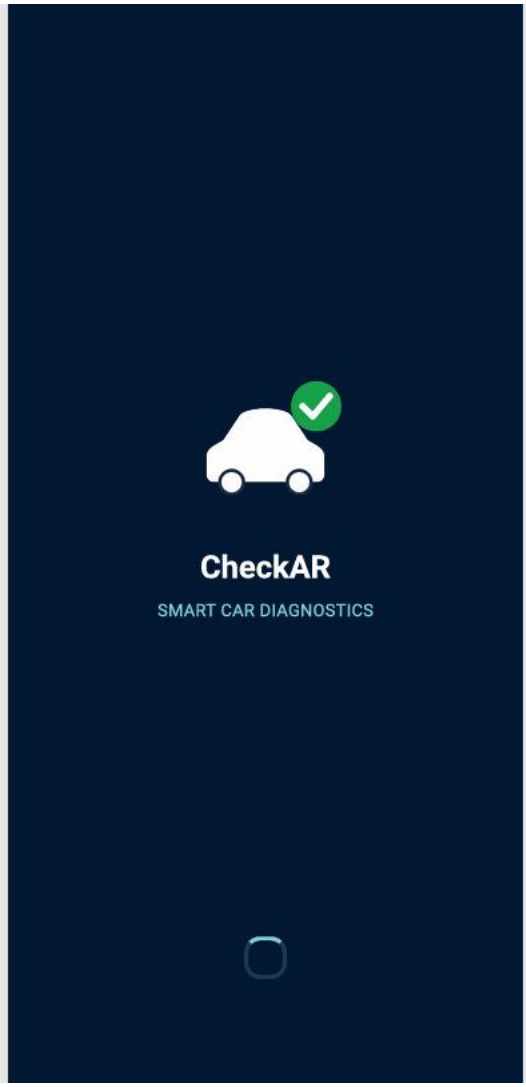
The logo design combines automotive and AI-inspired elements to symbolize intelligent diagnostics.

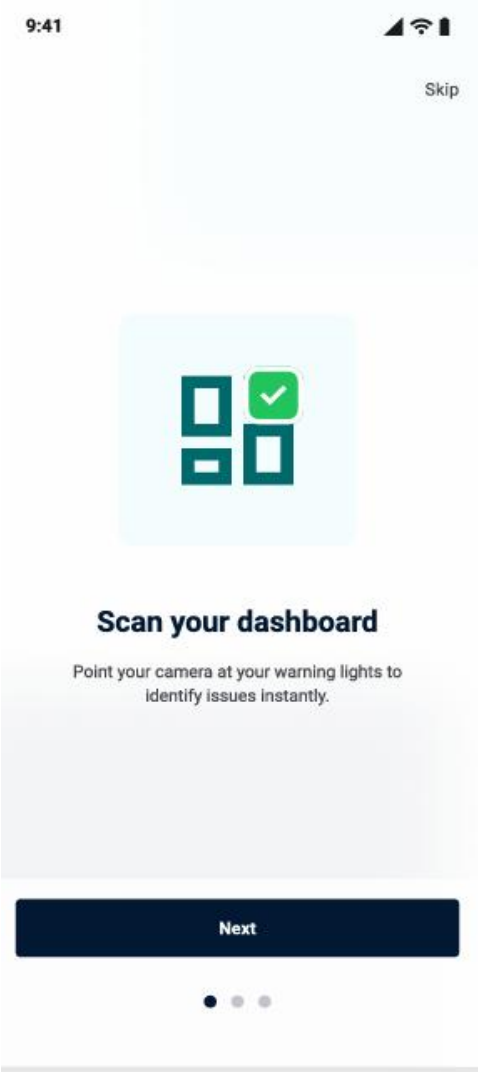
The visual identity communicates:

- Precision
- Reliability
- Modern technology
- Automotive intelligence

2.2.1 Logo Components Analysis

The logo consists of:





- A modern automotive symbol
- Circular diagnostic indicators
- Technology-inspired shapes
- Clean typography

These components reinforce the smart diagnostic concept.

2.3 Color Psychology and Primary Color Selection

2.3.1 Primary Color: Professional Blue (#2563EB)

Blue was selected because it represents:

- Trust
- Technology
- Stability
- Professionalism

It also improves readability and visual comfort.

2.3.2 Complete Color Palette

Color	**Purpose**
Blue (#2563EB)	Primary actions
White (#FFFFFF)	Background
Gray (#6B7280)	Secondary text
Red (#DC2626)	Critical alerts
Yellow (#FACC15)	Warning states
Green (#16A34A)	Healthy system status

2.3.3 Color Strategy Implementation

The color system was implemented to:

- Highlight urgency levels
- Improve navigation clarity
- Guide user attention
- Maintain consistent branding

3. Visual Design System

3.1 Typography Strategy

3.1.1 Primary Font Family: Inter

The Inter font family was selected because:

- It improves readability on mobile screens
- It supports modern UI design
- It provides excellent spacing and clarity

3.1.2 Typography Hierarchy

Element	Font Weight	Size
Headers	Bold	24px
Subheaders	Semi-Bold	18px
Body Text	Regular	14px
Buttons	Medium	16px

3.2 Layout and Spacing System

3.2.1 Grid System

The application uses:

- Responsive mobile grids
- Consistent alignment
- Adaptive screen scaling

3.2.2 Spacing Scale

A standardized spacing system improves consistency:

- 4px micro spacing
- 8px small spacing
- 16px medium spacing
- 24px large spacing

3.3 Iconography Standards

3.3.1 Icon Design Principles

Icons were designed to be:

- Minimalistic
- Easily recognizable
- Consistent in shape
- Scalable for different devices

3.3.2 Functional Icon Categories

The system includes:

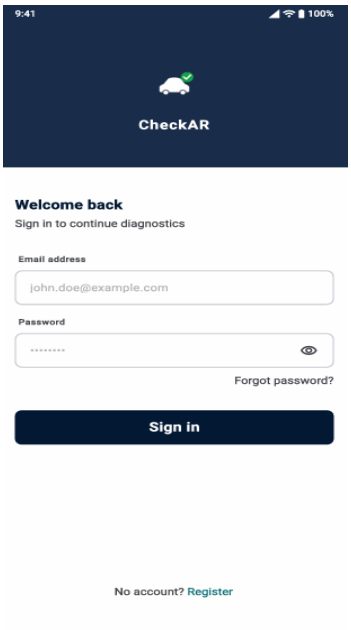
- Navigation icons
- Diagnostic icons
- Notification icons
- Vehicle status indicators
- Settings and profile icons

4. User Interface Design Specifications

4.1 Authentication Interface Design

4.1.1 Login Screen Components

The login screen includes:

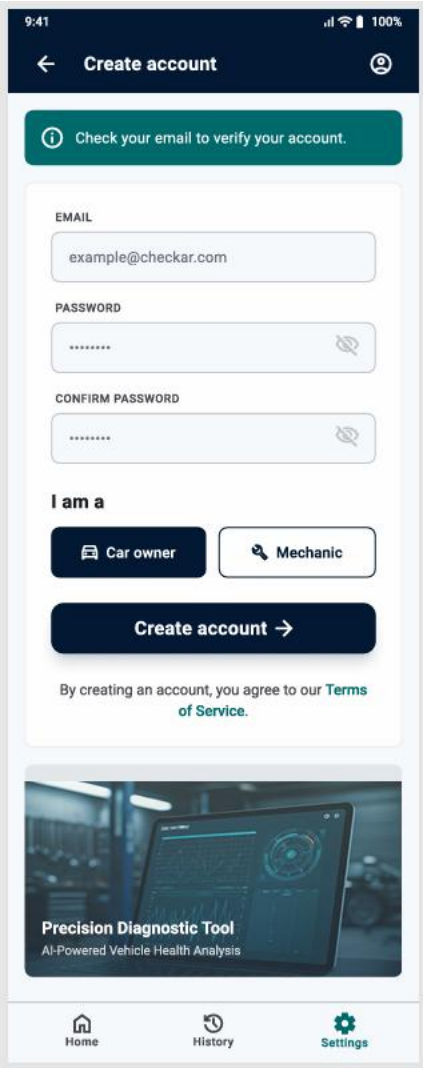


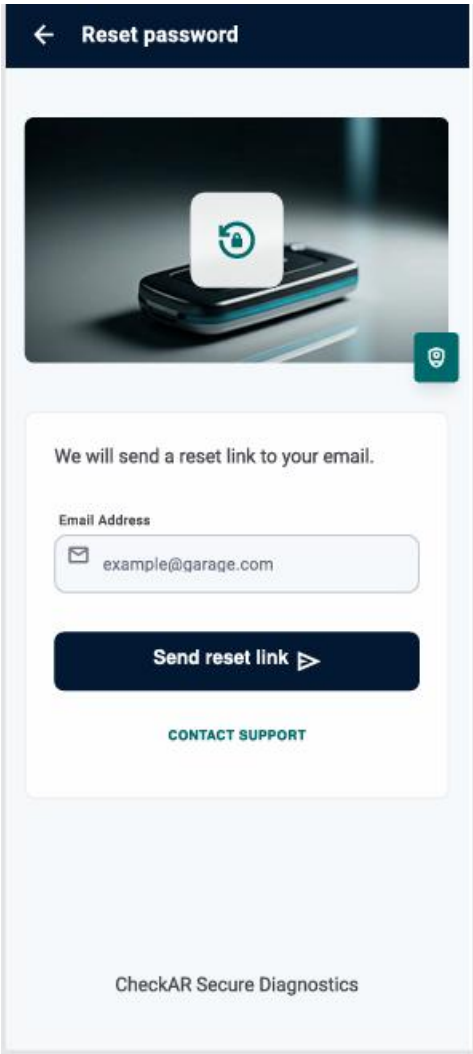
- Email field
- Password field
- Forgot password option
- Sign-in button
- Registration navigation

The interface emphasizes simplicity and fast authentication.

4.1.2 Registration Flow Design

The registration interface allows users to:





- Enter credentials
- Select user role
- Verify email

- Create secure accounts

4.1.3 Design Considerations

Special attention was given to:

- Security visibility
- Input validation
- User guidance
- Error prevention

4.2 Home Screen Interface

4.2.1 Layout Structure

The home screen includes:

- Greeting section
- Vehicle information card
- Diagnostic action buttons
- Recent diagnosis history
- Navigation bar

4.2.2 Visual Hierarchy

The interface prioritizes:

1. Diagnostic actions
2. Active warnings
3. Maintenance alerts
4. Historical reports

This ensures users quickly access critical functions.

4.3 Diagnostic Interface Design

4.3.1 Dashboard Scanner Interface

The scanner interface provides:

- Camera framing guides
- Lighting instructions
- Real-time scanning assistance

4.3.2 Sound Recorder Interface

The sound recording interface includes:

- Recording timer
- Positioning instructions
- Noise control guidance

4.3.3 Results Display Interface

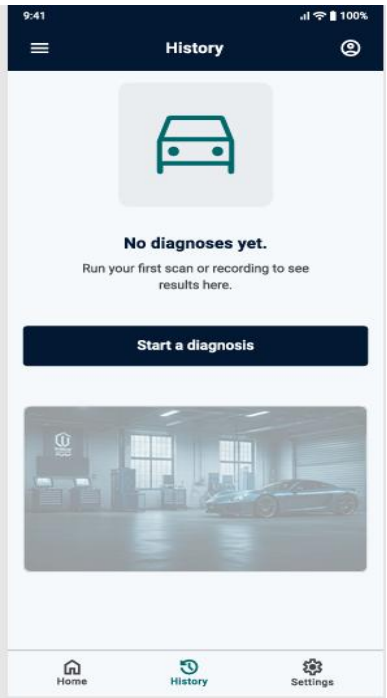
The results screen presents:

- Detected issues
- Severity levels
- Suggested actions
- Video tutorials
- Nearby mechanics

4.4 Maintenance Tracking Interface

4.4.1 History List Design

The history section organizes:



- Previous reports
- Dates and timestamps
- Issue categories
- Urgency levels

4.4.2 Reminder System Design

The reminder system uses:

- Color-coded alerts
- Scheduled notifications
- Maintenance tracking indicators

5. Frontend Implementation Strategy

5.1 Technology Stack Selection

5.1.1 Flutter Framework Advantages

Flutter was selected because it:

- Supports cross-platform development
- Provides fast UI rendering
- Offers rich widget libraries
- Improves development speed

5.1.2 State Management Architecture

The application uses Provider state management because:

- It simplifies data flow
- Improves scalability
- Enhances maintainability

5.2 Navigation Architecture

5.2.1 Navigation Structure

The application uses:

- Bottom navigation
- Named routes
- Screen-based navigation hierarchy

5.2.2 Navigation Implementation

Flutter Navigator APIs manage:

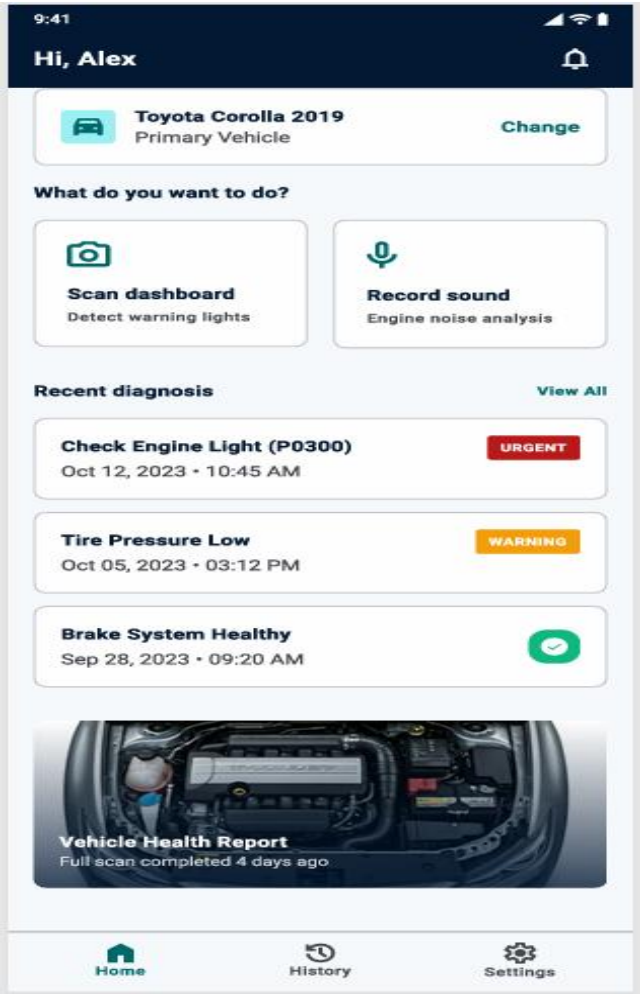
- Page transitions

- Navigation states
- Back-stack behavior

5.3 Screen Implementation Details

5.3.1 Home Screen (HomeScreen)

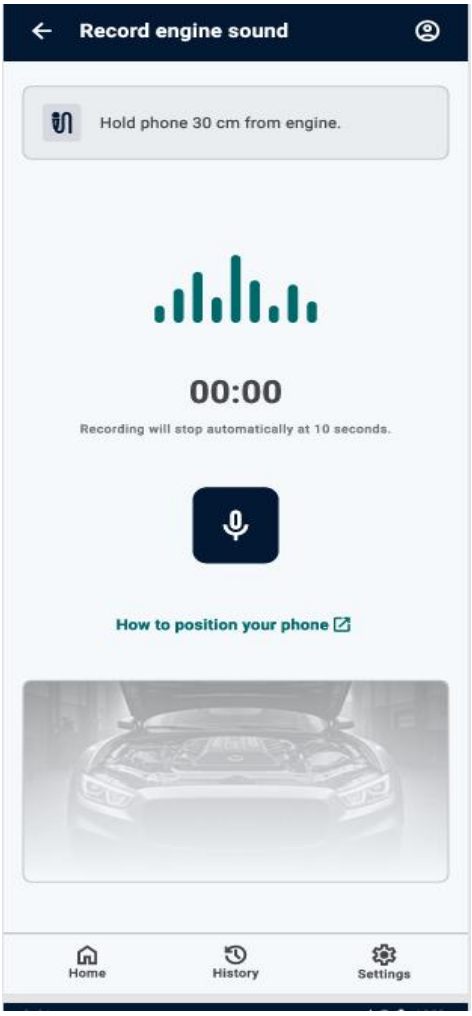
Functions include:

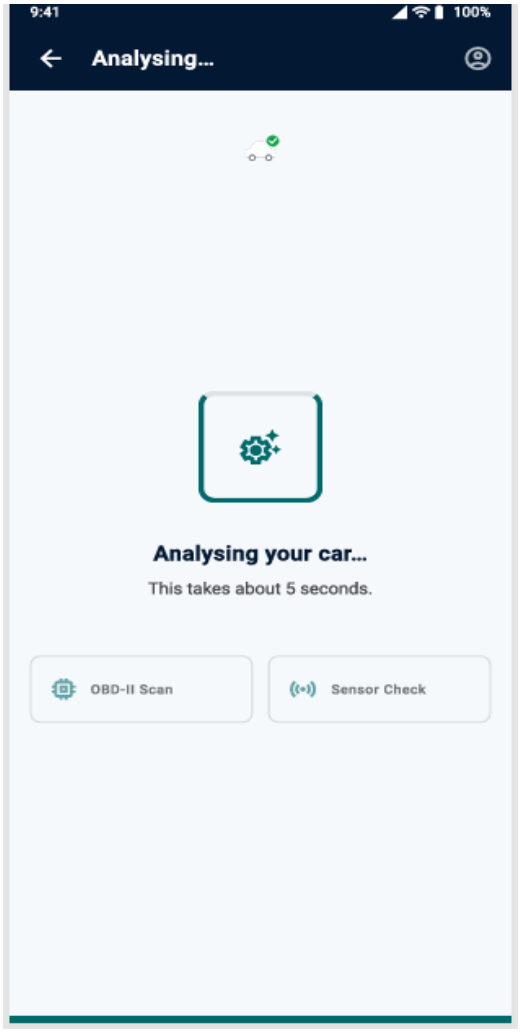


- Dashboard scanning access
- Engine recording access
- Diagnostic history display
- Health summary cards

5.3.2 Record Sound Screen (RecordSoundScreen)

Features include:

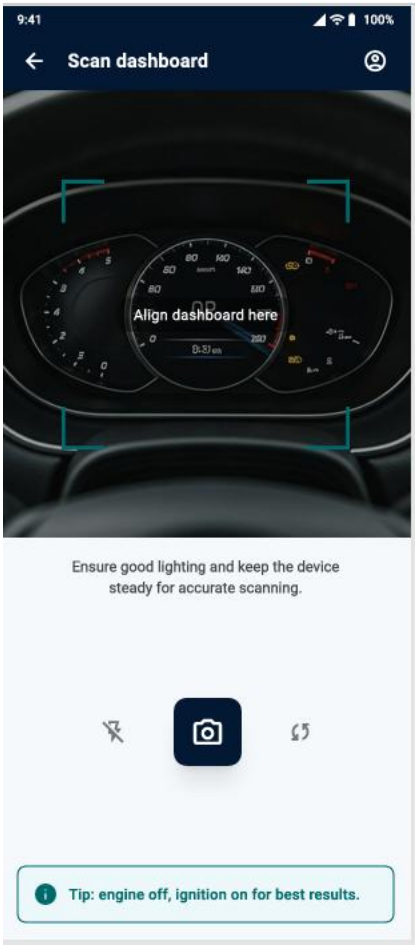




- Microphone permission handling
- Timer implementation
- Audio recording management
- Upload progress indicators

5.3.3 Scan Dashboard Screen (ScanDashboardScreen)

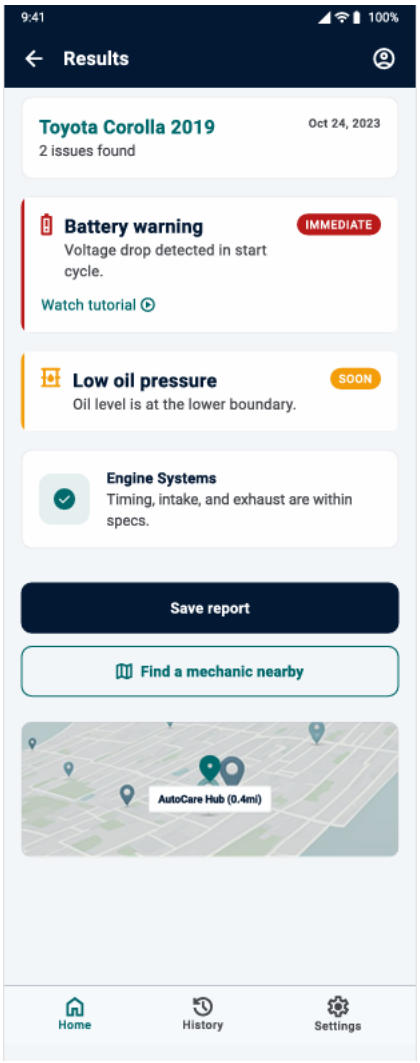
Includes:

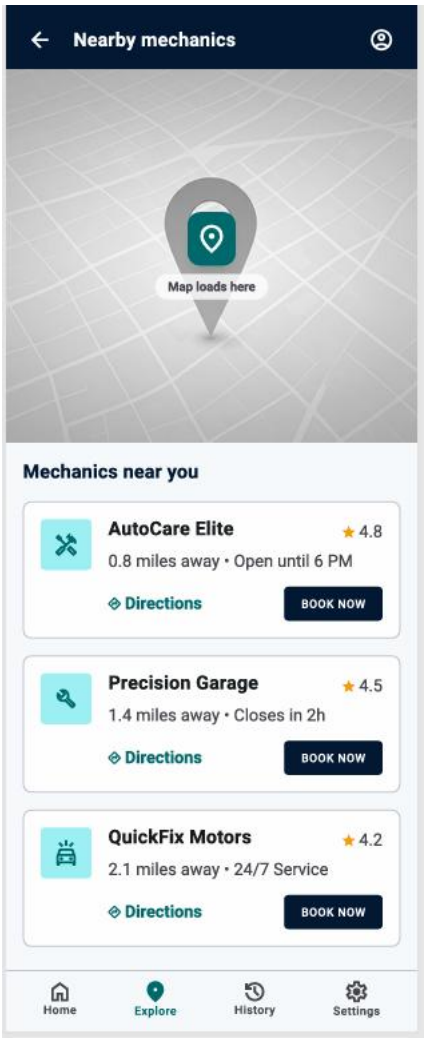


- Camera preview integration
- Frame alignment overlay
- Image capture functionality

5.3.4 Sound Diagnosis Result Screen (SoundDiagnosisResultScreen)

Displays:





- Fault analysis
- Severity indicators
- Repair recommendations
- Video tutorial links

5.3.5 Help & About Screen (HelpAboutScreen)

Contains:

- Application information
- Support contacts
- User guidelines
- FAQ sections

5.4 Technical Implementation Details

5.4.1 State Management

Provider architecture manages:

- Authentication state
- Diagnostic results
- User preferences
- Notification data

5.4.2 Asset Management

Assets include:

- Icons
- Images
- Audio resources
- Animation files

5.4.3 Package Integration

Key Flutter packages include:

- camera

- provider
- image picker
- audio recorder
- google_maps_flutter

5.4.4 Performance Considerations

Optimization techniques include:

- Lazy loading of screens
- Efficient image handling
- Proper disposal of controllers
    - Memory management for recordings

5.5 User Experience Features

5.5.1 Error Handling

The system handles:

- Camera errors
- Network issues
- Permission denials
- Invalid recordings

5.5.2 Loading States

Loading animations and progress indicators improve:

- Circular progress indicators
- Shimmer effects for loading states
- Clear feedback during analysis
    - Cancel options for long operations

5.5.3 Accessibility

Accessibility features include:

- Adequate touch targets (minimum 48x48px)
- Clear contrast ratios
- Descriptive button labels
    - Consistent navigation patterns

5.6 Responsive Design Implementation

5.6.1 Screen Size Adaptations

The application supports:

- Mobile-first design approach with progressive enhancement
- Flexible layouts using Flutter's responsive widget system
- Dynamic typography scaling based on device settings
    - Touch target optimization for various screen densities

5.6.2 Platform-Specific Considerations

The system adapts to:

- iOS design language compliance (Human Interface Guidelines)
- Android Material Design 3 implementation
- Platform-specific navigation patterns
    - Native integration points (sharing, notifications)

5.7 Performance Optimization Strategies

5.7.1 Rendering Performance

Performance was improved using:

- Efficient widget rebuilding strategies
- Image optimization and caching mechanisms
    - Lazy loading for large data sets (diagnostic history)

5.7.2 User Experience Optimization

Additional improvements include:

- Skeleton screens during content loading
- Progressive image enhancement
- Offline-first architecture with graceful degradation
    - Predictive preloading of frequently accessed content

6. Implementation Challenges and Solutions

6.1 Technical Challenges

6.1.1 ML Model Integration with UI Responsiveness

- **Challenge:** Maintaining UI responsiveness during heavy ML computations
- **Solution:** Asynchronous processing with loading states
- **Implementation:** Background isolates for heavy computations
    - **User Feedback:** Progress indicators and estimated completion times

6.1.2 Cross-Platform Consistency

- **Challenge:** Ensuring consistent behavior across iOS and Android
- **Solution:** Custom widget library with platform adaptations
- **Implementation:** Conditional rendering based on platform detection
- **Testing:** Comprehensive device matrix validation

6.1.3 Offline Functionality UI States

- **Challenge:** Managing UI states when offline
- **Solution:** Clear online/offline mode indicators
- **Implementation:** Sync status communication with user-friendly messaging
- **Fallback:** Graceful feature degradation with explanatory content

6.2 User Experience Challenges

6.2.1 Complex Information Presentation

- **Challenge:** Presenting complex diagnostic information clearly
- **Solution:** Progressive disclosure with layered information architecture
- **Implementation:** Expandable cards with summary-to-detail flow
- **Validation:** User testing with target demographic groups

6.2.2 Trust Building through Design

- **Challenge:** Building user trust in diagnostic accuracy
- **Solution:** Consistent professional visual language
- **Implementation:** Credibility indicators and transparent information sourcing
- **Measurement:** User confidence surveys and retention metrics

7. Testing and Validation Strategy

7.1 Usability Testing Framework

7.1.1 Testing Phases

The testing process included:

1. **Prototype Testing:** Early design validation with wireframes
2. **Alpha Testing:** Internal team evaluation with functional prototypes
3. **Beta Testing:** Limited external user group evaluation
4. **Production Testing:** Continuous user feedback collection

7.1.2 Key Metrics

Measured metrics included:

- Task completion rates for primary user flows
- Time-to-completion for diagnostic processes
- Error rates and recovery success
- User satisfaction scores (SUS - System Usability Scale)

7.2 A/B Testing Strategy

7.2.1 Testing Variables

Tested variables included:

- Button placements
- Navigation layouts
- Color schemes
- Diagnostic card designs

7.2.2 Success Criteria

Success was evaluated using:

- Reduced interaction time

- Increased usability scores
- Higher completion rates
- Improved conversion from installation to active usage

- Higher user retention rates

7.3 Accessibility and Inclusive Design

7.3.1 WCAG 2.1 AA Compliance

The design follows:

- Contrast standards
- Readability standards
- Accessible navigation principles

7.3.2 Inclusive Design Features

Features include:

- Simplified language
- Responsive scaling
- Visual indicators
- Accessible touch controls
- **Internationalization Support:**
    - RTL (Right-to-Left) language support architecture
    - Flexible text scaling without layout breaking
    - Cultural color sensitivity considerations
    - Localized content organization patterns
- **Motor Accessibility:**
    - Generous touch targets (minimum 44px)
    - Drag-and-drop alternatives for complex interactions
    - Voice command integration potential
    - One-handed operation considerations

8. Future Enhancement Roadmap

8.1 Phase 2 Design Improvements

8.1.1 Advanced Personalization

Future improvements include:

- Personalized vehicle dashboards
- Smart maintenance recommendations
- AI-driven preferences

8.1.2 Enhanced Visualization

Additional features may include:

- 3D dashboard visualization
- Real-time engine analytics
- Interactive repair tutorials

8.2 Scalability Considerations

8.2.1 Design System Evolution

The design system will support:

- Future feature expansion
- Modular UI components
- Scalable frontend architecture

8.2.2 International Expansion

Future versions may support:

- Multiple languages
- Regional mechanic databases
- International vehicle standards

9. Conclusion

The CheckAR Smart Car Diagnostics Application successfully combines modern UI/UX principles with intelligent automotive diagnostics to create an accessible and user-friendly mobile platform.

The frontend implementation provides a responsive, visually appealing, and intuitive experience that supports both technical and non-technical users. Through the use of Flutter, responsive design systems, accessibility principles, and optimized performance strategies, the application delivers efficient and scalable vehicle diagnostic functionality.

The UI design not only enhances usability but also strengthens user confidence in AI-powered diagnostics through clear communication, structured navigation, and professional visual presentation.

Overall, the project demonstrates how thoughtful interface design and frontend engineering can simplify complex automotive diagnostics and improve the driving maintenance experience for users.

References

1. Software Requirements Specification (SRS) - ..\Desktop\Group11SchoolWork\Task 3\SRS.pdf
2. Flutter Documentation - UI Development Best Practices
3. Material Design 3 - Google Design System Guidelines
4. Human Interface Guidelines - Apple iOS Design Standards
5. Web Content Accessibility Guidelines (WCAG) 2.1
6. Nielsen Norman Group - Mobile UX Design Principles
7. Automotive User Interface Design Standards - ISO 15005
8. Color Psychology in User Interface Design - Academic Research Compilation

9. Figma design wireframe file - https://www.figma.com/design/AsjO9aj80QJPleykoCJpRf/checkAR?node-id=0-1&t=iFhe38gzWwNR0biw-1

10. Github Repository for frontend implementation https://github.com/kelseafunwi/Group11SchoolWork